ЛАБОРАТОРНАЯ 1 · ВАРИАНТ «СВЕТОФОР»

Разбираемся простыми словами

Как работает твоя программа и что отвечать на защите. Сначала объяснения с нуля, затем 80 вопросов с короткими ответами.

Что нужно сделать в этой работе

Нужно заставить плату управлять светофором: переключать лампы по времени и реагировать на кнопку. Ещё нужно написать свои функции для работы с выводами микроконтроллера.

Плата - это устройство, на котором установлены микроконтроллер, кнопка и светодиоды. **Микроконтроллер** - маленький компьютер, который выполняет твою программу.

Самая главная мысль

Программа всё время быстро проверяет две вещи: нажали ли кнопку и сколько времени прошло. Когда время нужной фазы закончилось, она переключает свет. Пока время не вышло, она продолжает проверять кнопку.

Что можно сказать в начале защиты

«Я сделал светофор на микроконтроллере STM32F427. Он переключает красный, зелёный, мигающий зелёный и жёлтый. Кнопка может сократить ожидание красного. Чтобы одно нажатие не считалось несколько раз, я добавил проверку устойчивого нажатия. Программа постоянно проверяет кнопку и время. Для работы с выводами я написал свой драйвер».

Как учить

Сначала прочитай объяснения. Потом попробуй ответить на вопросы своими словами, закрыв ответы. Вопросы 1-50 дают основу. Вопросы 51-80 помогут, если преподаватель начнёт подробнее спрашивать о коде и устройстве микроконтроллера.

Разобрана текущая реализация в папке lab1. Официального списка вопросов преподавателя в проекте нет.

1. Что к чему подключено

Вывод, порт и GPIO

Вывод, или пин, - электрический контакт микроконтроллера. Через одни контакты программа получает сигналы, через другие управляет устройствами.

Порт - группа таких выводов. Например, PD13 означает вывод 13 порта D. Это его имя в программе; номер ножки на корпусе может быть другим.

GPIO - сокращённое название обычных цифровых входов и выходов. Проще говоря, через GPIO можно прочитать кнопку или включить светодиод.

Что подключено	Куда	Что делает программа
Зелёный светодиод	PD13	Подаёт 1, чтобы включить
Двухцветный светодиод	PD14 и PD15	Выбирает красный или жёлтый
Кнопка	PC15	Читает 0 или 1

Что означают 0 и 1

Это два цифровых уровня: низкий и высокий. На этой плате высокий уровень обычно близок к 3,3 В, низкий - к 0 В. Для программы они выглядят как 1 и 0.

Но смысл зависит от подключения. Зелёный светодиод включается от 1. А **нажатая кнопка читается как 0**. Поэтому в программе перед чтением кнопки стоит !: он меняет 0 на true, то есть «нажата».

Как выбирается красный или жёлтый

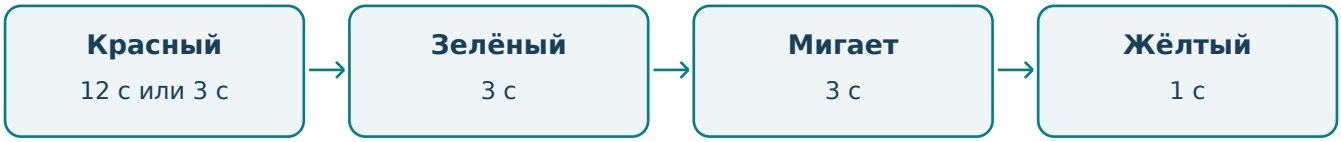
Это один индикатор с двумя цветами. Он включён между PD14 и PD15. PD15=1 и PD14=0 дают красный. PD15=0 и PD14=1 дают жёлтый. Если уровни одинаковые, индикатор не горит.

Как это объяснить на защите

«Светодиоды подключены к выходам, поэтому программа задаёт на них уровни. Кнопка подключена ко входу, поэтому программа читает её уровень».

2. Как светофор выбирает следующий свет

У программы есть четыре **состояния**. Состояние - это просто ответ на вопрос: «Что светофор делает сейчас?»



После жёлтого снова включается красный.

В коде такой способ организации называется **конечным автоматом**. Не нужно представлять что-то сложное: программа хранит название текущей фазы и знает, какую включить следующей.

Что делает кнопка

Когда нажали	Что произойдёт
На красном	Текущий красный будет длиться 3 с от своего начала вместо 12 с
На обычном зелёном	Это нажатие игнорируется
Во время мигания	Программа запомнит запрос: ближайший красный будет коротким
На жёлтом	Программа тоже запомнит запрос на ближайший красный

Самый важный пример

Красный включился. Через 1 секунду программа подтвердила нажатие. Зелёный включится на отметке 3 секунды от начала красного. Значит, после нажатия осталось около 2 секунд.

Если нажатие подтвердилось через 8 секунд красного, зелёный включится сразу: нужные 3 секунды уже прошли.

Короткий красный - это 3 секунды от начала красного, а не ещё 3 секунды после нажатия.

Откуда взялись 12 и 3 секунды

В коде обычный красный в четыре раза длиннее обычного зелёного: 12 = 4 × 3. После просьбы красный сокращается до четверти: 12 / 4 = 3. Мигание - отдельная фаза.

Что значит «запомнить запрос»

У программы есть переменная `request`. В ней хранится «да» или «нет»: просил ли пользователь сократить красный. После выхода из красного просьба считается выполненной и стирается.

Во всех примерах речь о нажатии, которое программа уже подтвердила после проверки кнопки. Почему это нужно - на следующей странице.

3. Почему одно нажатие нужно проверять

Что такое дребезг

Внутри кнопки соприкасаются металлические контакты. Когда ты нажимаешь, они могут несколько раз быстро соединиться и разойтись, прежде чем окончательно замкнутся.

Человек нажал один раз, а программа без проверки может увидеть несколько нажатий. Такие быстрые колебания называют **дребезгом контактов**.

Как это исправлено в твоём коде

Когда программа замечает новое положение кнопки, она ещё не считает его окончательным. Сначала ждёт, чтобы наблюдаемый уровень не менялся 30 миллисекунд. 30 мс - это 0,03 секунды.

Если уровень снова изменился, отсчёт начинается заново. Если новое положение подтвердилось, программа принимает его. Та же проверка работает при отпускании.

Пример по времени

Время	Что заметила программа
100 мс	Кнопка как будто нажата. Начали проверку.
104 мс	Уже отпущена. Счёт начинаем заново.
108 мс	Снова нажата. Ещё раз начинаем отсчёт.
138 мс	Прошло 30 мс без наблюдаемых изменений. Нажатие принято.

Почему удержание не даёт много нажатий

Программа различает **«кнопка сейчас нажата»** и **«кнопку только что нажали»**. Светофору передаётся второй вариант: одно событие в момент подтверждения.

Если держать кнопку десять секунд, это всё ещё одно нажатие. Для следующего нужно отпустить её, дождаться подтверждения отпускания и нажать заново.

Что сказать на защите

«Я проверяю, что новое положение кнопки сохраняется 30 мс. Поэтому короткие колебания не считаются отдельными нажатиями. Удержание тоже не создаёт повторов».

4. Как программа следит за временем

Представь секундомер

В программе есть счётчик миллисекунд. Одна секунда - это 1000 миллисекунд. Функция `HAL_GetTick()` позволяет посмотреть текущее значение счётчика.

Когда начинается красный, программа запоминает показание счётчика. Затем сравнивает его с новым показанием. Разница и есть прошедшее время.

```
прошло = время_сейчас - время_начала
```

Например, красный начался при значении 5000. Сейчас счётчик показывает 8000. Значит, прошло 3000 мс, то есть 3 секунды.

Почему нет обычной паузы на 12 секунд

Если поставить в основном цикле длинную паузу, этот цикл перестанет проверять кнопку на всё время ожидания. Пользователь нажмёт, а программа может этого не заметить.

Поэтому программа не ждёт на одном месте. Она снова и снова смотрит: «Уже пора переключить свет?» Если ещё рано, она продолжает опрашивать кнопку.

Кто увеличивает счётчик

В микроконтроллере есть таймер **SysTick**. Он подаёт сигнал раз в миллисекунду. Процессор ненадолго выполняет функцию `SysTick_Handler`, которая увеличивает счётчик времени, затем возвращается к основному коду.

Такой короткий переход к специальной функции называется **прерыванием**. Поэтому правильно говорить: «Кнопку я проверяю в цикле, а прерывание SysTick использую для отсчёта времени».

Как получается мигание

Во время мигающего зелёного лампа горит полсекунды, потом полсекунды не горит. Полный повтор занимает одну секунду. Сама фаза мигания длится три секунды.

Тонкости переполнения счётчика вынесены в вопросы 57-58. Для понимания основной программы сначала достаточно примера с секундомером.

5. Что такое регистры и драйвер

Регистр - небольшая ячейка управления

Чтобы настроить вывод микроконтроллера, программа записывает числа в специальные ячейки - **регистры**. Они связаны с устройством напрямую: запись может поменять режим вывода или его уровень.

У каждого регистра своя задача. Один выбирает режим, другой позволяет прочитать вход, третий меняет выход. Названия стоит знать, но учить их проще вместе со смыслом.

Название	Простой смысл
RCC_AHB1ENR	Разрешить работу нужного порта: включить его тактирование
MODER	Выбрать, что делает вывод: вход, выход или другая функция
IDR	Прочитать цифровой уровень на выводе
ODR	Посмотреть, какой выходной уровень задан программой
BSRR	Включить или выключить выбранные выходные биты одной записью
PUPDR	Выбрать внутреннюю подтяжку к питанию или земле
OTYPER	Выбрать, как выход формирует электрический сигнал
OSPEEDR	Выбрать, насколько быстро меняется напряжение на выходе
AFR / LCKR	Выбрать другую функцию вывода / защитить его настройки

Драйвер - функции, которые делают эту работу

Например, вместо ручной настройки всех регистров в main вызывается `io_configure(...)`. Эта функция проверяет параметры и записывает настройки в нужном порядке. Набор таких функций и называют **драйвером**.

Твой GPIO-драйвер написан самостоятельно. Готовая библиотека HAL используется для запуска и времени. Функции HAL_GPIO для управления выводами твоё приложение не вызывает.

Как сказать проще

«Регистры управляют аппаратурой. Мой драйвер - это функции, которые правильно читают и изменяют эти регистры».

6. Настройки вывода на понятных примерах

Вход и выход

Вход: микроконтроллер слушает внешний сигнал. Так подключена кнопка. **Выход:** микроконтроллер сам задаёт уровень. Так он управляет лампами.

Подтяжка: чтобы отпущенный вход имел ясный уровень

Отпущенная кнопка не должна оставлять вход без определённого уровня. Поэтому резистор соединяет его с питанием. Пока кнопка отпущена, читается 1; при нажатии она соединяет линию с землёй, и читается 0.

Такой резистор называют **подтяжкой к питанию**, или pull-up. На стенде он уже есть снаружи микроконтроллера. Поэтому внутренняя подтяжка для кнопки выключена.

Push-pull и open-drain

Push-pull: выход может сам установить и 0, и 1. В этой работе так управляются лампы.

Open-drain: выход умеет сам установить только 0. Когда программа задаёт 1, он перестаёт удерживать линию. Для получения высокого уровня нужна подтяжка. Этот режим поддерживает драйвер, но для ламп здесь он не выбран.

Альтернативная функция

Один контакт может использоваться по-разному. Например, им может управлять передатчик UART - устройство для обмена данными. Тогда контакт работает в режиме **альтернативной функции**, сокращённо AF. Для кнопки и ламп в ЛР 1 такой режим не нужен.

Скорость выхода - это не скорость мигания

Настройка OSPEEDR определяет, насколько быстро напряжение переходит между 0 и 1. Период мигания задаёт программа. Можно включить медленный выход и всё равно переключать лампу раз в полсекунды - так здесь и сделано.

Для чего резистор у светодиода

Он ограничивает ток. Слишком большой ток может повредить светодиод или выход микроконтроллера. На стенде необходимые резисторы уже установлены.

7. Если спросят про биты и маски

Эту страницу удобно разбирать с листом бумаги. Все операции сначала показаны на маленьких числах.

Бит - одна цифра, которая может быть только 0 или 1. В регистрах GPIO биты связаны с выводами. Чтобы выбрать нужный вывод, программа отмечает соответствующую позицию единицей.

Сдвиг <<: поставить единицу на нужное место

```
1 << 0  = 0001
1 << 1  = 0010
1 << 2  = 0100
1 << 3  = 1000
```

Позиции считают с нуля, справа налево. Поэтому `1U << 13` отмечает вывод 13. Это и есть маска одного вывода.

Символ |: объединить выбранные позиции

```
0001 | 0100 = 0101
```

Так выбираем сразу два вывода: 0 и 2. В работе через | объединены маски выводов 13, 14 и 15 - всех трёх линий ламп.

Символ &: оставить только нужные позиции

```
1101 & 0100 = 0100
```

Слева - прочитанное значение, справа - маска. Остался только выбранный бит. Если результат равен нулю, в выбранных позициях единиц не было.

Символ ~: поменять все биты на противоположные

Из единиц получаются нули, из нулей - единицы. Вместе с маской это помогает выбрать выводы, которые нужно выключить. Число битов зависит от типа числа в С.

Как это используется в BSRR

Сначала драйвер определяет, какие выходы поставить в 1, а какие - в 0. Затем объединяет оба списка в одной команде: нижние 16 бит задают единицы, верхние 16 - нули.

Для красного нужно включить PD15 и выключить PD13/PD14. Команда получается `0x60008000`. Начало `0x` означает запись числа в шестнадцатеричной системе. Здесь главное понять смысл: одна команда задаёт все три уровня и не трогает посторонние выводы.

8. Как читать твой код

Главный цикл состоит из четырёх понятных действий

Код	Что происходит
HAL_GetTick()	Посмотреть время
button_update(...)	Проверить, появилось ли новое подтверждённое нажатие
traffic_update(...)	Решить, пора ли менять фазу светофора
traffic_light(...) и led_write(...)	Выбрать нужный свет и подать уровни на выводы

Перевод названий переменных

Переменная	Как её понимать
traffic.state	Какой сейчас этап: красный, зелёный, мигание или жёлтый
traffic.since	Когда начался этот этап
traffic.request	Есть ли просьба сократить красный
button.candidate	Какое положение кнопки увидели последним
button.pressed	Какое положение уже подтвердили
button.changed_at	Когда замечено последнее изменение

Пример чтения условия

```
if (press && t->state != TRAFFIC_GREEN)
    t->request = true;
```

Читаем вслух: «Если появилось нажатие и сейчас не обычный зелёный, запомнить просьбу сократить красный». && означает «и», != - «не равно», true - «да».

Где искать нужную часть

`main.c` соединяет всё вместе. `traffic.c` отвечает за поведение светофора. `button.c` проверяет кнопку. `led.c` управляет группой ламп. `gpio_driver.c` работает с регистрами. В `board.h` записано, к каким выводам всё подключено.

Вопросы 1-10. Самое главное

Ответы сформулированы так, чтобы их было удобно произнести на защите.

1. Что ты сделал в этой работе?

Светофор, который переключает лампы по времени и реагирует на кнопку. Для работы с выводами микроконтроллера я написал свои функции.

2. Что такое микроконтроллер?

Это маленький компьютер в одной микросхеме: у него есть процессор, память и устройства для работы с внешними сигналами.

3. Какой микроконтроллер здесь используется?

STM32F427 на учебном стенде SDK-1.1MC.427. Он выполняет программу и управляет светодиодами.

4. Что такое GPIO?

Это обычные цифровые входы и выходы микроконтроллера. Через них можно читать кнопку и управлять светодиодами.

5. Чем вход отличается от выхода?

Через вход программа узнаёт внешний уровень. Через выход она сама задаёт уровень для внешнего устройства.

6. Что означает PD13?

Вывод 13 порта D. Буква выбирает группу выводов, число - вывод внутри этой группы.

7. Куда подключены светодиоды?

Зелёный подключён к PD13. Красный и жёлтый выбираются уровнями на PD14 и PD15.

8. Куда подключена кнопка?

К PC15. Когда она отпущена, программа читает 1. Когда нажата - 0.

9. Какой порядок работы светофора?

Красный, обычный зелёный, мигающий зелёный, жёлтый. Затем цикл повторяется с красного.

10. Сколько длится каждая фаза?

Красный обычно 12 секунд, обычный зелёный 3 секунды, мигание ещё 3 секунды, жёлтый 1 секунду.

Вопросы 11-20. Поведение светофора

11. Зачем нужна кнопка?

Она позволяет попросить программу сократить ожидание красного. Точное действие зависит от того, какая фаза сейчас идёт.

12. Что происходит при нажатии на красном?

Красный должен закончиться через 3 секунды от своего начала вместо обычных 12 секунд.

13. Нажали через секунду красного. Сколько ещё ждать?

Около двух секунд: вместе получится три секунды от начала красного. Считаем момент, когда программа уже подтвердила нажатие.

14. Нажали через восемь секунд красного. Что будет?

Зелёный включится сразу после подтверждения нажатия. Три секунды от начала красного уже прошли.

15. Что будет при нажатии на обычном зелёном?

Это нажатие игнорируется. Обычный зелёный продолжает свою фазу.

16. Что будет при нажатии во время мигания?

Программа запомнит просьбу. Мигание закончится как обычно, а ближайший красный будет длиться три секунды.

17. Что будет при нажатии на жёлтом?

Программа тоже запомнит просьбу. Жёлтый не сокращается, зато следующий красный будет коротким.

18. Зачем нужна переменная request?

Она хранит, поступила ли просьба сократить красный. Это одно значение «да» или «нет».

19. Несколько нажатий заново запускают красный?

Нет. Программа сохраняет просьбу, но время начала красного не меняет. Несколько просьб не складываются в очередь.

20. Кнопка сокращает все следующие красные?

Нет. Просьба используется один раз и стирается при выходе из красного. Для следующего сокращения нужно новое подходящее нажатие.

Вопросы 21-30. Кнопка без загадок

21. Почему нажатая кнопка даёт 0?

Так она подключена: нажатие соединяет линию с землёй. При отпускании внешний резистор возвращает высокий уровень.

22. Что делает ! перед чтением кнопки?

Меняет логическое значение на противоположное. Из физического нуля получается понятное программе «кнопка нажата».

23. Что такое дребезг?

Контакты кнопки при одном нажатии могут несколько раз быстро замкнуться и разомкнуться. Программа должна считать это одним нажатием.

24. Как ты убираешь влияние дребезга?

Принимаю новое положение кнопки, только если наблюдаемый уровень не менялся 30 миллисекунд.

25. Почему выбрано 30 мс?

Это выбранный в программе порог проверки. Он отсекает короткие колебания; для другой кнопки его можно подобрать заново.

26. От какого момента считаются эти 30 мс?

От последней замеченной смены уровня. Если контакт снова изменился, отсчёт начинается заново.

27. Что будет, если держать кнопку?

Будет одно событие нажатия. Пока кнопку не отпустили, удержание не создаёт новых событий.

28. Проверяется ли отпускание?

Да. Отпускание тоже должно подтвердиться за 30 мс. После него можно зарегистрировать следующее нажатие.

29. Что будет, если включить плату с зажатой кнопкой?

Программа примет это за начальное положение, а не новое нажатие. Нужно отпустить кнопку и нажать снова.

30. А если нажать очень быстро?

Нажатие короче 30 мс может не пройти проверку и будет проигнорировано. Очень редкий опрос тоже может пропустить короткое нажатие.

Вопросы 31-40. Время и порядок действий

31. Как программа узнаёт, сколько прошло времени?

Читает счётчик миллисекунд и вычитает сохранённое время начала фазы. Полученная разница - длительность текущей фазы.

32. Что делает HAL_GetTick?

Возвращает текущее значение счётчика времени. Здесь одна единица счётчика соответствует одной миллисекунде.

33. Почему в цикле нет HAL_Delay?

Во время длинной паузы основной цикл не проверял бы кнопку. Вместо паузы программа постоянно проверяет прошедшее время.

34. Что такое SysTick?

Таймер внутри процессора. В этой программе он раз в миллисекунду вызывает короткую функцию, которая увеличивает счётчик времени.

35. Используются ли прерывания?

Да, для SysTick. Кнопка проверяется обычным чтением в основном цикле; отдельного прерывания от неё здесь нет.

36. Как часто мигает зелёный?

Полсекунды горит, полсекунды не горит. Полный повтор занимает одну секунду, то есть частота мигания равна 1 Гц.

37. Что такое состояние программы?

Это сохранённый ответ на вопрос «какая фаза сейчас идёт?». Например, TRAFFIC_RED означает красный.

38. Почему это называют конечным автоматом?

У программы есть ограниченный набор состояний и правила перехода между ними. Здесь четыре фазы и переходы по времени.

39. Как программа одновременно следит за кнопкой и светом?

Она быстро делает короткие действия по очереди: проверяет кнопку, время, фазу и выходы. За счёт повторения цикла всё выглядит одновременным.

40. Почему время читается один раз за проход цикла?

Чтобы кнопка и светофор использовали одно и то же показание времени. Так проще согласовать их действия.

Вопросы 41-50. Регистры и файлы

41. Что такое регистр?

Специальная ячейка, через которую программа управляет аппаратурой или узнаёт её состояние. Например, читает уровень кнопки.

42. Что такое драйвер?

Набор функций для работы с устройством. Здесь драйвер GPIO настраивает выводы и читает или меняет их уровни.

43. Зачем нужен MODER?

Он выбирает режим вывода: например, вход для кнопки или выход для светодиода.

44. Чем IDR отличается от ODR?

IDR показывает цифровой уровень на самом выводе. ODR хранит уровень, который программа задала выходу. Это не всегда одно и то же.

45. Зачем нужен BSRR?

Он позволяет одной записью включить одни выходные биты и выключить другие. Поэтому лампы переключаются одной командой.

46. Зачем включать тактирование порта?

Это разрешает работу соответствующего блока GPIO. В коде сначала включается нужный порт, затем настраиваются его выводы.

47. Что находится в board.h?

Пины и комбинации уровней для конкретного стенда. Например, какие выводы относятся к лампам и кнопке.

48. За что отвечает traffic.c?

За правила светофора: текущую фазу, её время, просьбу пользователя и выбор нужного света.

49. За что отвечают button.c и led.c?

button.c проверяет нажатие и дребезг. led.c выдаёт нужную комбинацию уровней на группу светодиодов.

50. Для чего всё разделено на файлы?

Чтобы каждая часть решала свою задачу. Например, правила светофора можно менять отдельно от проверки кнопки.

Вопросы 51-60. Если спросят подробнее

Эти ответы тоже можно объяснить обычными словами. Техническое название указано рядом со смыслом.

51. Что такое битовая маска?

Число, которое отмечает нужные выводы единицами. Маска помогает менять только лампы и сохранять остальные выводы порта.

52. Что означает 1U << 13?

Берём число с одной единицей и сдвигаем её на 13 позиций. Получаем маску вывода 13. Буква U означает беззнаковое число.

53. Как маска выбирает все три лампы?

В ней отмечены биты 13, 14 и 15. В коде такая маска равна 0xE000; это запись числа в шестнадцатеричной системе.

54. Что значит «атомарная запись» в BSRR?

Изменение нужных выходных битов выполняется одной аппаратной записью. Между отдельными программными командами включения и выключения ламп нет промежутка.

55. Как BSRR отличает включение бита от выключения?

Нижние 16 бит команды устанавливают выходные биты в 1, верхние 16 сбрасывают их в 0. Нулевые биты команды ничего не меняют.

56. Почему нельзя записать только нужный цвет прямо во весь ODR?

Так можно случайно изменить другие выводы того же порта. Собственный драйвер использует маску и BSRR, чтобы трогать только выбранную группу.

57. Что будет, когда счётчик времени достигнет максимума?

Он начнёт считать с нуля. Так бывает примерно раз в 49,7 суток. Программа учитывает это при вычислении разницы времени.

58. Почему now - since работает через такой переход?

Используются беззнаковые числа: после максимума они продолжаются с нуля. Для наших коротких интервалов и регулярной проверки разность остаётся правильной.

59. Что будет, если главный цикл надолго задержится?

Свет переключится при следующей проверке. Новая фаза начнётся с этого момента и получит своё полное время. Пропущенные фазы программа не прокручивает подряд.

60. Сколько длится весь цикл?

Обычно 19 секунд: 12 + 3 + 3 + 1. С коротким красным - 10 секунд: 3 + 3 + 3 + 1. Возможны небольшие задержки выполнения.

Вопросы 61-70. Настройки и собственный драйвер

61. Что такое pull-up и зачем он кнопке?

Это подтяжка к питанию через резистор. Она задаёт единицу, когда кнопка отпущена. На стенде такой резистор уже есть снаружи микроконтроллера.

62. Что означает push-pull?

Выход сам формирует и низкий, и высокий уровни. Такой режим выбран для ламп.

63. Что означает open-drain?

Выход сам может удерживать только ноль. При команде 1 он отпускает линию; высокий уровень создаётся подтяжкой.

64. Меняет ли OSPEEDR частоту мигания?

Нет. Он настраивает скорость электрического перехода между уровнями. Полсекунды между переключениями отсчитывает сама программа.

65. Что такое альтернативная функция, или AF?

Режим, в котором выводом управляет другое устройство внутри микроконтроллера, например передатчик UART. Для ламп и кнопки в этой работе AF не используется.

66. Зачем задавать уровень до включения выходного режима?

Чтобы при включении выхода сразу появился нужный уровень. Иначе на короткое время может появиться старое значение и возникнуть лишний импульс.

67. Что означает IO_HANDOFF_REQUIRED?

Драйвер сообщает: сначала нужно подготовить смену владельца вывода. Если им управляет другая периферия, её нужно остановить и освободить вывод перед новой настройкой.

68. Зачем проверять LCKR?

Этот регистр может защищать настройки вывода от изменения. Если защита включена, драйвер возвращает IO_LOCKED и отказывается от настройки.

69. Что будет при неверных параметрах io_configure?

Функция вернёт IO_INVALID. Проверки выполняются до записей в GPIO, чтобы ошибка не оставила выбранную группу настроенной наполовину.

70. Почему драйвер собственный, если используются CMSIS и HAL?

CMSIS даёт готовые имена и описания регистров, HAL здесь помогает с запуском и временем. Настройку и чтение/запись GPIO выполняют мои собственные функции.

Вопросы 71-80. Код, запуск и проверка

71. Чем HAL отличается от LL?

HAL - более готовые функции для работы с микроконтроллером. LL - функции, которые ближе к отдельным регистрам. Собственный GPIO-драйвер в этой работе не вызывает ни HAL_GPIO, ни LL_GPIO.

72. HAL всегда ставит программу на паузу?

Нет. Это зависит от конкретной функции. Например, HAL_Delay ждёт, а HAL_GPIO_WritePin меняет выход и тоже использует BSRR.

73. Для чего volatile?

Сообщает компилятору, что значение может измениться извне или обращение к нему важно для аппаратуры. Например, счётчик времени меняется в прерывании.

74. Можно ли безопасно настраивать порт из двух мест сразу?

Текущий драйвер на это не рассчитан. Если два участка прерывают друг друга и меняют общие настройки, нужно отдельно согласовать доступ. Одна запись BSRR не защищает все остальные действия.

75. Что делают перед вызовом main?

Подготавливают стек - память для вызовов функций, задают начальные значения переменных и выполняют стартовую настройку. Потом запускают main.

76. Откуда берётся частота работы микроконтроллера?

Здесь используется внутренний генератор HSI на 16 МГц. Для кнопки и ламп этого достаточно. Внешний кварц для работы программы не нужен.

77. Какой отладчик используется на стенде?

Встроенный адаптер FT2232 с интерфейсом JTAG. Его настройки для OpenOCD записаны в SDK11M.cfg. Это нужно учитывать при подключении к плате.

78. Что смотреть, если программа не работает?

Сначала выполняется ли main и меняется ли счётчик времени. Затем - фазу светофора, состояние кнопки, уровни GPIO и lab1_fault: в нём сохраняется код серьёзной ошибки.

79. Что проверяют программные тесты?

Поведение кнопки, переходы светофора, точные границы времени, переход счётчика через ноль и команды GPIO. Они проверяют код, но не измеряют напряжения и токи реальной платы.

80. Что можно улучшить в будущем?

Например, проверить сигналы на реальной плате измерительными приборами или уменьшить расход энергии. Но сначала нужно показать, что текущая программа правильно выполняет все свои правила.

Проверяем себя на примерах

Попробуй сначала ответить сам. Речь о моменте, когда программа уже подтвердила нажатие.

Ситуация	Ответ и объяснение
Прошла 1 с красного, нажали кнопку	Зелёный будет через 2 с. Красный суммарно длится 3 с.
Прошло 8 с красного, нажали кнопку	Зелёный сразу. Порог в 3 с уже пройден.
Нажали на мигающем зелёном	Мигание закончится, пройдёт жёлтый, затем будет красный на 3 с.
Нажали на жёлтом	Жёлтый закончится как обычно. Следующий красный короткий.
Нажали на обычном зелёном и держим	Если нажатие подтвердилось на обычном зелёном, оно игнорируется. Удержание нового события не даст.
Несколько раз нажали на красном	Отсчёт не начнётся заново. Просьба всё равно одна.
Ровно 3 с красного и есть просьба	Можно включать зелёный: порог уже достигнут.
Кнопка зажата при включении	Нужно отпустить её и нажать заново.

Небольшая тонкость с концом зелёного

Важно, когда программа подтвердила нажатие, а не только когда палец коснулся кнопки. Если проверка 30 мс закончилась уже во время мигания, просьба будет принята.

Если нажатие подтвердилось прямо в том вызове, который заканчивает обычный зелёный, оно ещё относится к обычному зелёному и игнорируется. В коде сначала обрабатывается нажатие, затем меняется фаза.

Задача на мигание

С начала мигания прошло 200 мс - зелёный горит. Прошло 700 мс - не горит. Прошло 1200 мс - снова горит. Объяснение: он меняет состояние каждые 500 мс.

Что повторить прямо перед защитой

Пять вещей, которые нужно объяснить уверенно

1. Общая работа. Светофор проходит четыре фазы. Кнопка может сократить красный.

2. Время. Программа сравнивает показания счётчика, а основной цикл продолжает проверять кнопку.

3. Кнопка. Её нажатие даёт 0. Новое положение проверяется 30 мс. Удержание не считается новым нажатием.

4. GPIO. Выходы управляют лампами, вход читает кнопку. Драйвер работает через регистры.

5. Самый важный пример. Нажали после 1 секунды красного - осталось примерно 2 секунды. Нажали после 8 секунд - зелёный сразу после подтверждения.

Если попросят показать это в коде

Что показать	Где находится
Какие пины используются	Core/Inc/board.h
Длительности фаз	Core/Inc/traffic.h
Основной цикл	Core/Src/main.c
Правила кнопки и светофора	Core/Src/button.c и traffic.c
Работа с регистрами	Core/Src/gpio_driver.c

Проверка и источники

В предыдущем разборе сборка прошивки и 85 проверок C-кода прошли успешно. При упрощении памятки программа не менялась. Работа реального стенда в этой проверке не подтверждалась.

Памятка опирается на исходники lab1, README, docs/DEFENSE.md, тесты и схему docs/SDK1.1M.pdf. В технических вопросах полезны руководства ST: RM0090 о регистрах, DS9405 о микроконтроллере, UM1725 о библиотеках HAL и LL.

RM0090: описание регистров

DS9405: STM32F427

UM1725: HAL и LL